B tree, B+ tree

B+ Tree

- A B+ Tree is a balanced tree data structure used in DBMS for indexing and searching records efficiently. It helps the database retrieve records quickly without scanning the entire table.
- In a B+ Tree:

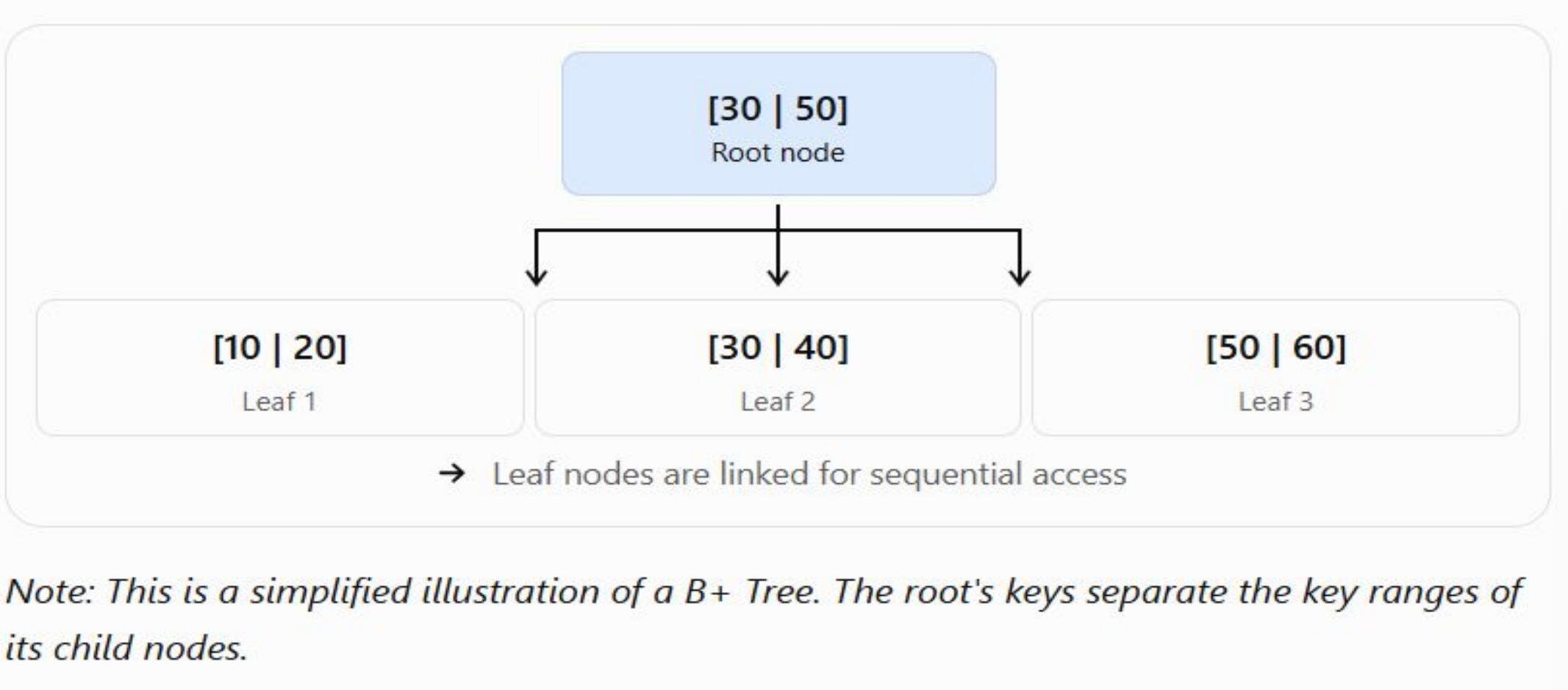
Internal nodes store search keys and pointers to child nodes.

Leaf nodes store all the search keys and pointers to the actual records.

All leaf nodes are at the same level, so the tree remains balanced.

Leaf nodes are linked together, making range queries and sequential access faster.

Structure of a B+ Tree



Note: This is a simplified illustration of a B+ Tree. The root's keys separate the key ranges of its child nodes.

Example: Constructing a B+ Tree

- Suppose a database stores student records using their roll numbers as index keys.
- Insert the following keys in ascending order:
- 10, 20, 30, 40, 50, 60
- Assume the order is 4, with a maximum of 3 keys per node.

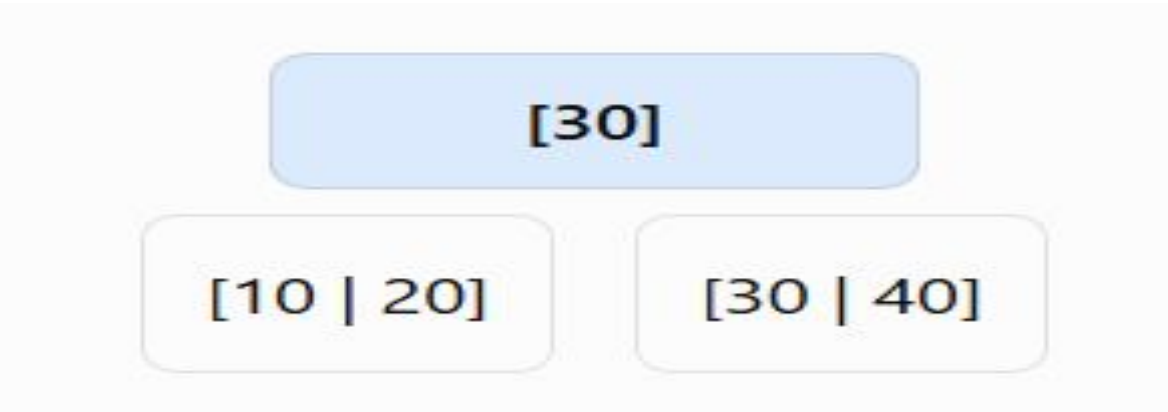
Step 1: Insert 10, 20 and 30

Initially, all keys fit in a single leaf node.



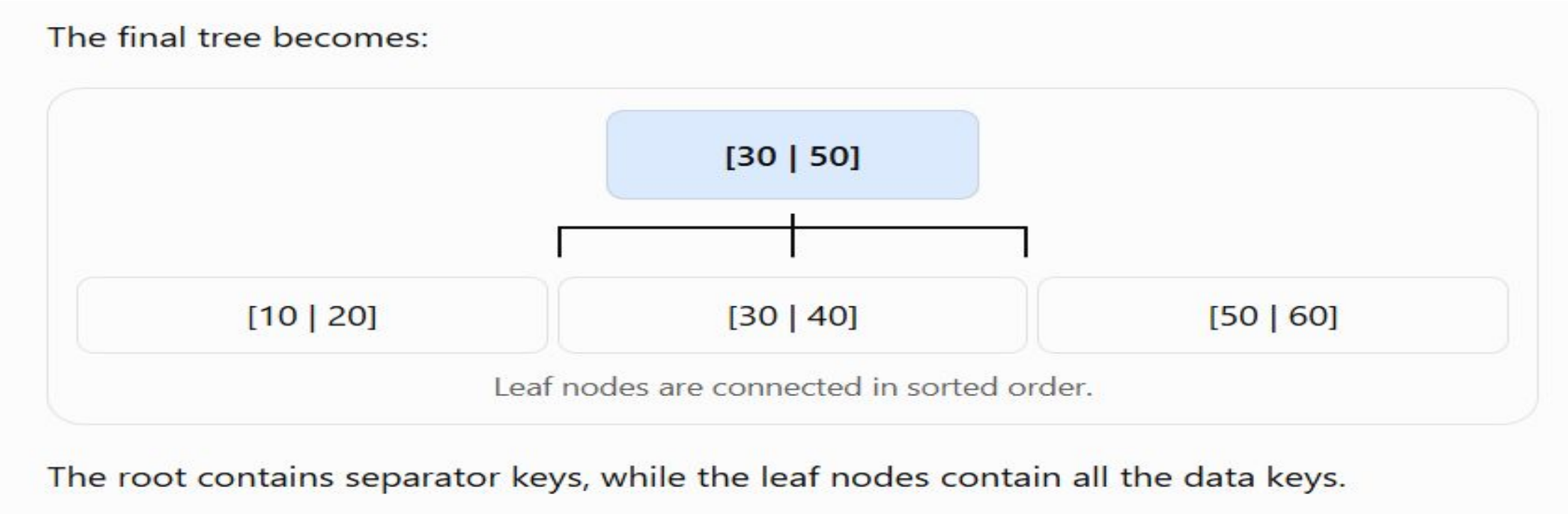
Step 2: Insert 40

- The leaf node exceeds its capacity, so it splits into two leaf nodes. The first key of the new right leaf is copied to the parent as a separator.
- The root key 30 directs searches to the correct leaf.



Step 3: Insert 50 and 60

- The right leaf eventually overflows again and splits. The separator key 50 is inserted into the root.



How does searching work?

- Suppose we want to search for roll number 40.
- Start at the root node [30 | 50].
- Since 40 is greater than or equal to 30 and less than 50, follow the middle child pointer.
- Reach the leaf node [30 | 40].
- Find key 40 and follow its record pointer to retrieve the student's record.
- Search path



Deletion

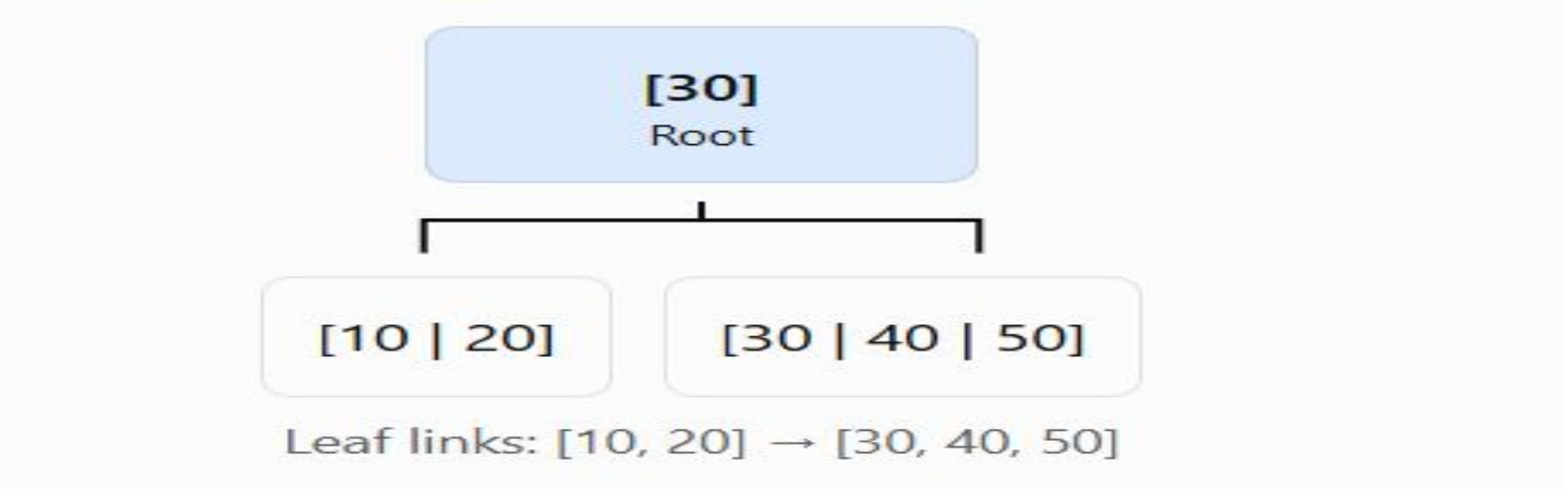
- In a B+ Tree, deleting a key may cause a node to have too few keys. The tree fixes this using:
- Redistribution (borrowing): Move a key from a sibling that has enough keys.
- Merging: Combine two nodes and remove the corresponding separator from their parent.

Step 1: Delete 60

- Initially, the tree is:
- Root: [30 | 50]
- Leaves: [10 | 20], [30 | 40], [50 | 60]
- Remove 60 from the last leaf.
- The last leaf becomes [50], which has only one key. This violates the minimum of two keys. Its left sibling [30 | 40] has only two keys, so it cannot lend one.

Solution: Merge the two rightmost leaves.

- Combine [30 | 40] and [50] into [30 | 40 | 50], then remove separator 50 from the root.



Step 2: Delete 10

- Starting with the tree obtained after Step 1:
- Root: [30]
- Leaves: [10 | 20] and [30 | 40 | 50]
- Delete 10. The left leaf becomes [20], which has fewer than two keys.
- This time, the right sibling [30 | 40 | 50] has three keys, so it can lend one.

Solution: Redistribute (borrow)

- Move 30 from the right leaf to the left leaf. The leaves become [20 | 30] and [40 | 50]. Update the root separator from 30 to 40.



Advantages of B+ Trees

- Fast searching: Balanced structure reduces the number of disk accesses.
- Efficient insertion and deletion: Nodes split, merge, or redistribute keys when needed.
- Range queries: Linked leaf nodes support queries such as roll numbers between 20 and 50.
- Sequential access: Records can be traversed in sorted order.
- Suitable for large databases: High branching factor keeps the tree height small.

B-Tree vs. B+ Tree

Feature	B-Tree	B+ Tree
Data pointers	May appear in internal and leaf nodes	Stored at leaf level
Internal nodes	Keys and possibly record pointers	Keys and child pointers
Leaf nodes linked	Not necessarily	Yes, typically
Range queries	Less convenient	More efficient
Database indexing	Used in some systems	Widely used for database indexes

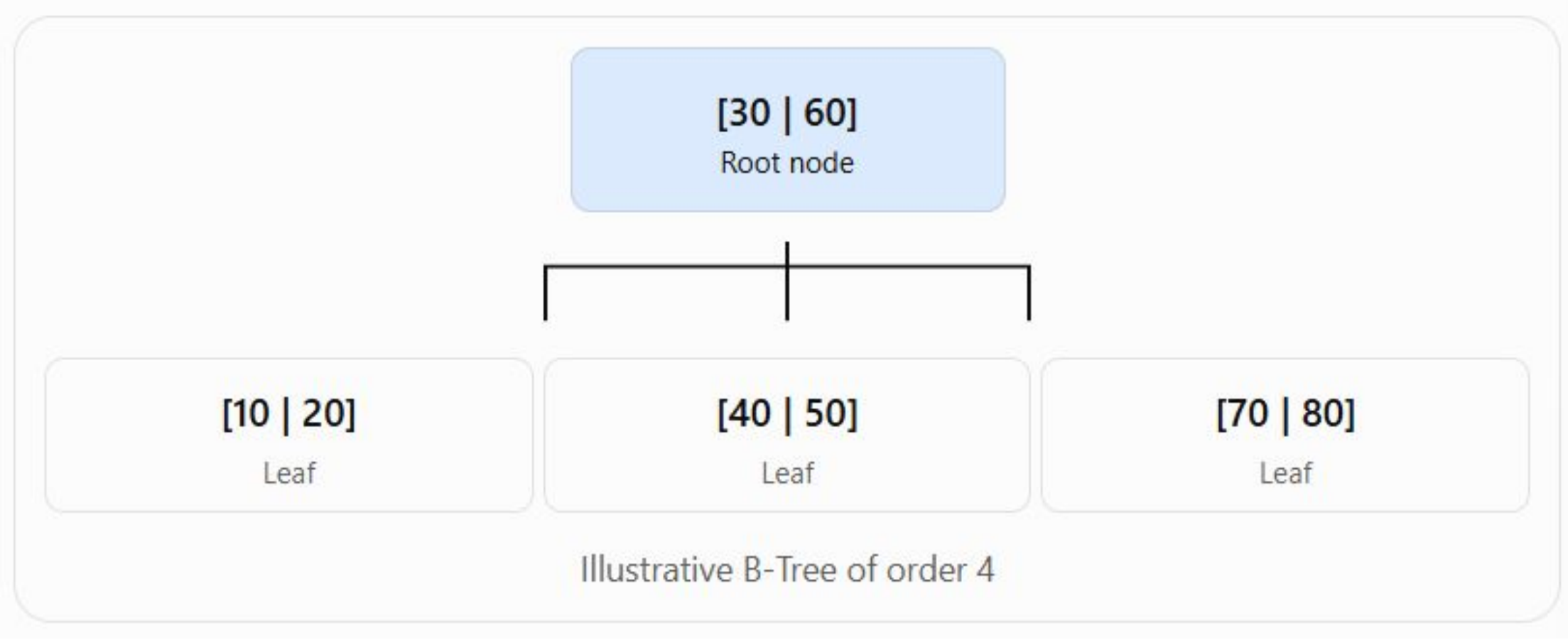
B-Tree

- A B-Tree (Balanced Tree) is a self-balancing search tree used in DBMS to store and retrieve records efficiently. It keeps keys in sorted order and allows searching, insertion, and deletion with fewer disk accesses.
- **Main characteristics**
- Keys are stored in sorted order within each node.
- Each node can contain multiple keys and child pointers.
- All leaf nodes are at the same level.
- The tree remains balanced after insertions and deletions.
- Searching can finish at either an internal node or a leaf node.

Structure of a B-Tree

- Consider a B-Tree of order 4.
- Maximum children per node = 4
- Maximum keys per node = 3
- Minimum children for a non-root internal node = 2
- Minimum keys for a non-root node = 1
- Important: In a B-Tree, keys in internal nodes can themselves point to records. The keys are not necessarily repeated in the leaves, unlike in a B+ Tree.

Structure of a B-Tree



Constructing a B-Tree

- Let us insert the following keys into a B-Tree of order 4:
- 10, 20, 30, 40, 50, 60, 70

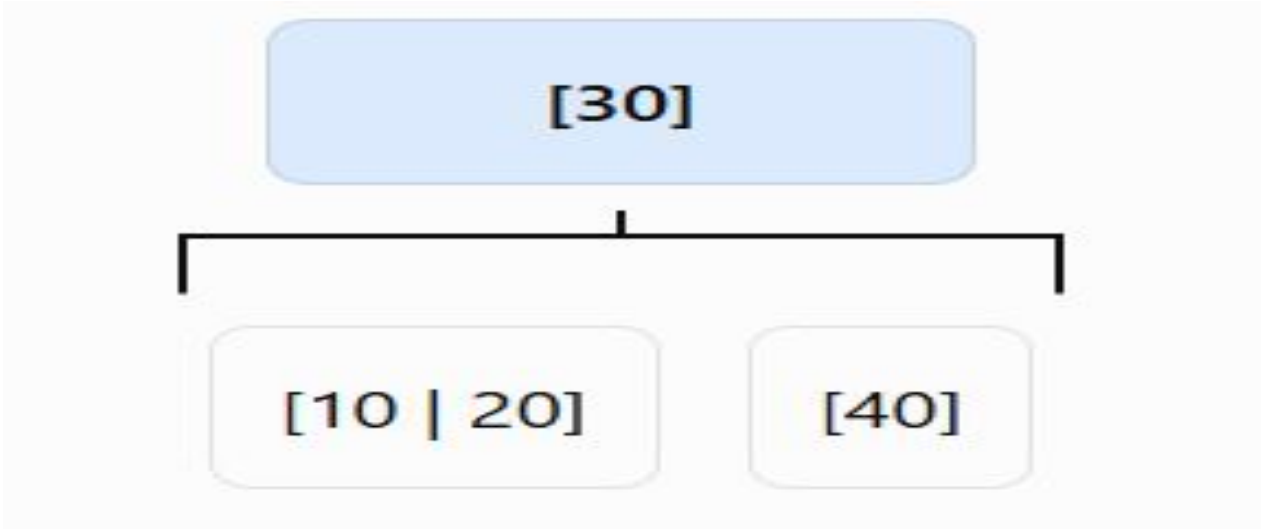
Step 1: Insert 10, 20 and 30

- All three keys fit in a single node.



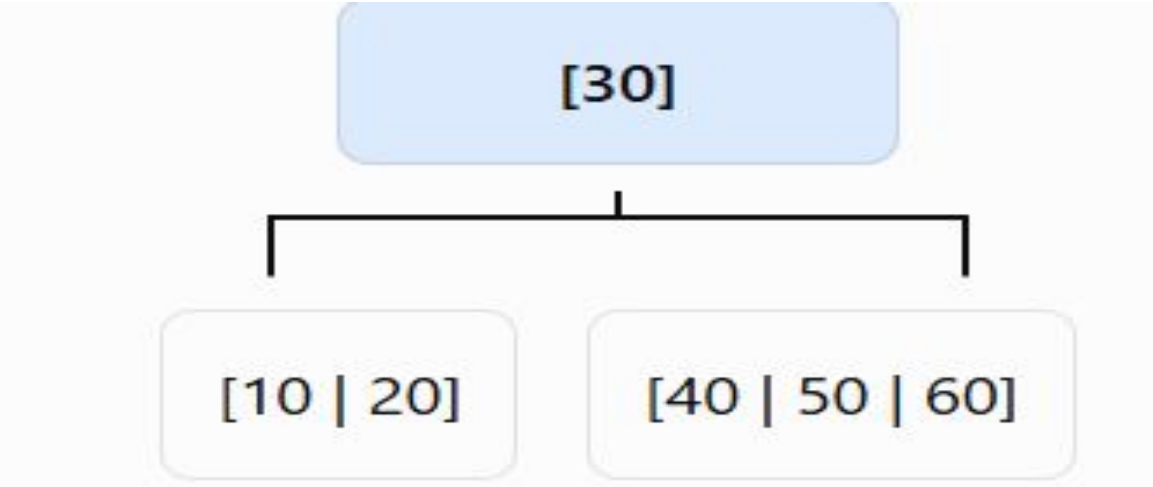
Insert 40

- The node now contains four keys, exceeding the maximum of three. Split the node and promote the middle key, 30, to become the root.



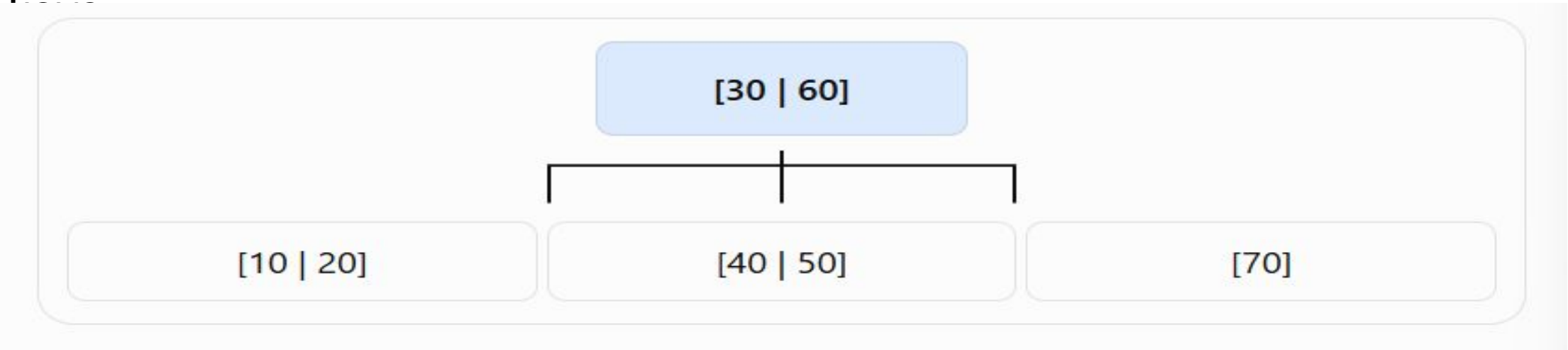
Step 3: Insert 50 and 60

- The keys 50 and 60 are inserted into the right child.



Step 4: Insert 70

- The right child would contain four keys: 40, 50, 60, 70. It overflows, so split it and promote 60 to the root.
- The tree is balanced, and each node respects the maximum of three



Searching in a B-Tree

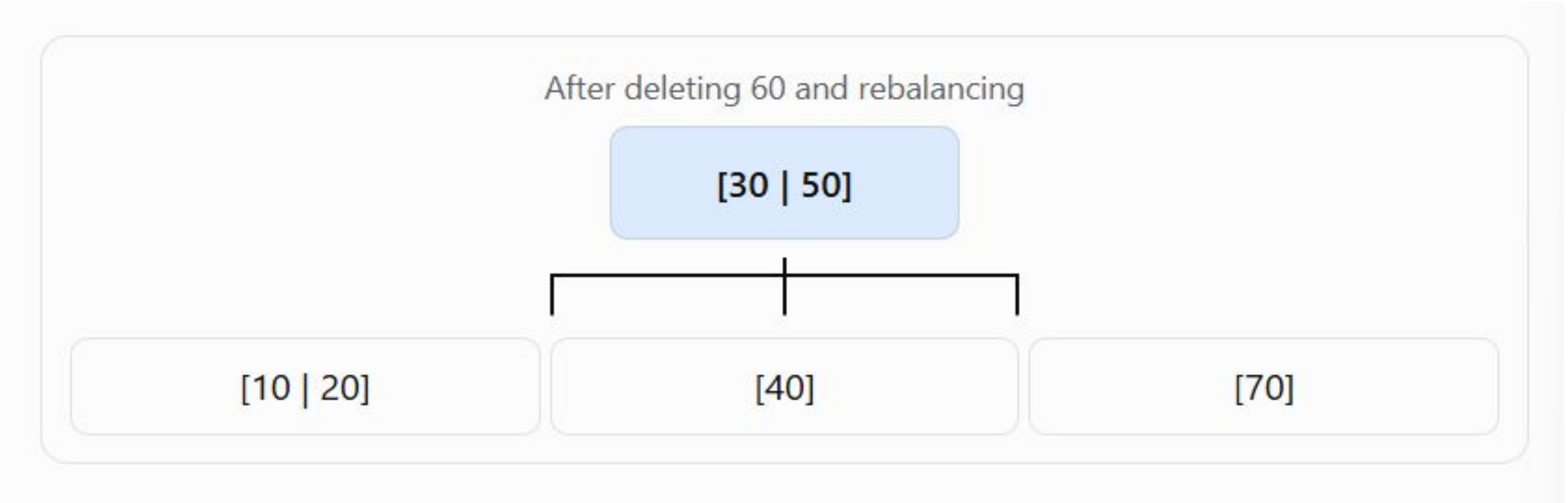
- Suppose we want to search for key 50.
- Start at the root [30 | 60].
- Since 50 is greater than 30 and less than 60, follow the middle child pointer.
- Reach [40 | 50].
- Find 50 in that node and retrieve the associated record pointer.



Step 1: Delete 60

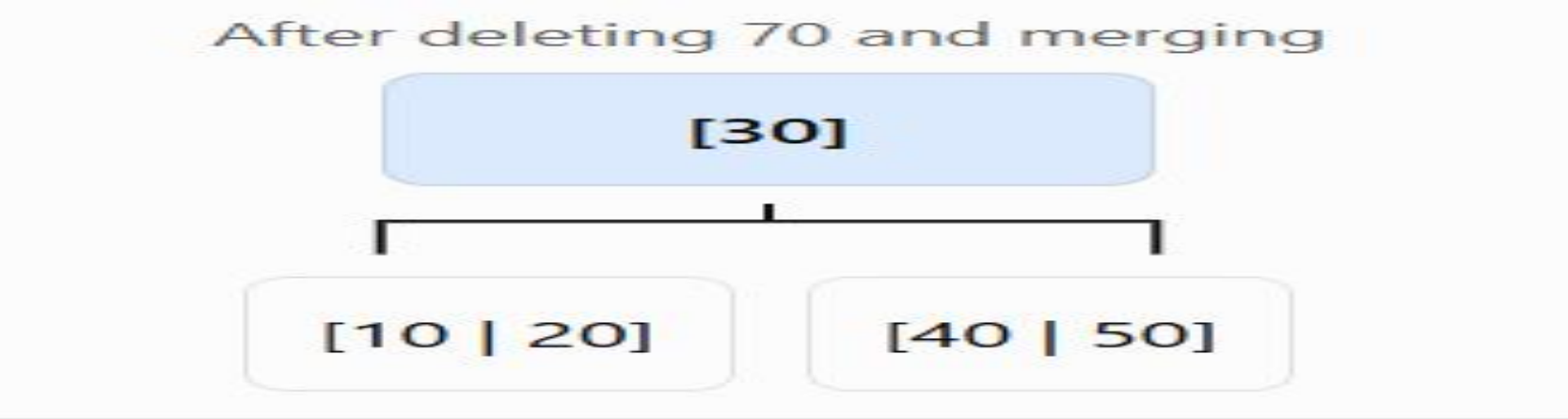
- In a B-Tree, when deleting a key from an internal node, we can replace it with its in-order predecessor or in-order successor, then delete the replacement key from its original node.
- Here, choose the in-order successor of 60, which is 70.
- Replace 60 with 70 in the root.
- Delete 70 from the right leaf [70].
- The right leaf becomes empty, so it underflows.
- The left sibling [40 | 50] has two keys and can lend one. Move 50 up to the root and move 70 down to the right child.

Step 1: Delete 60



Step 2: Delete 70

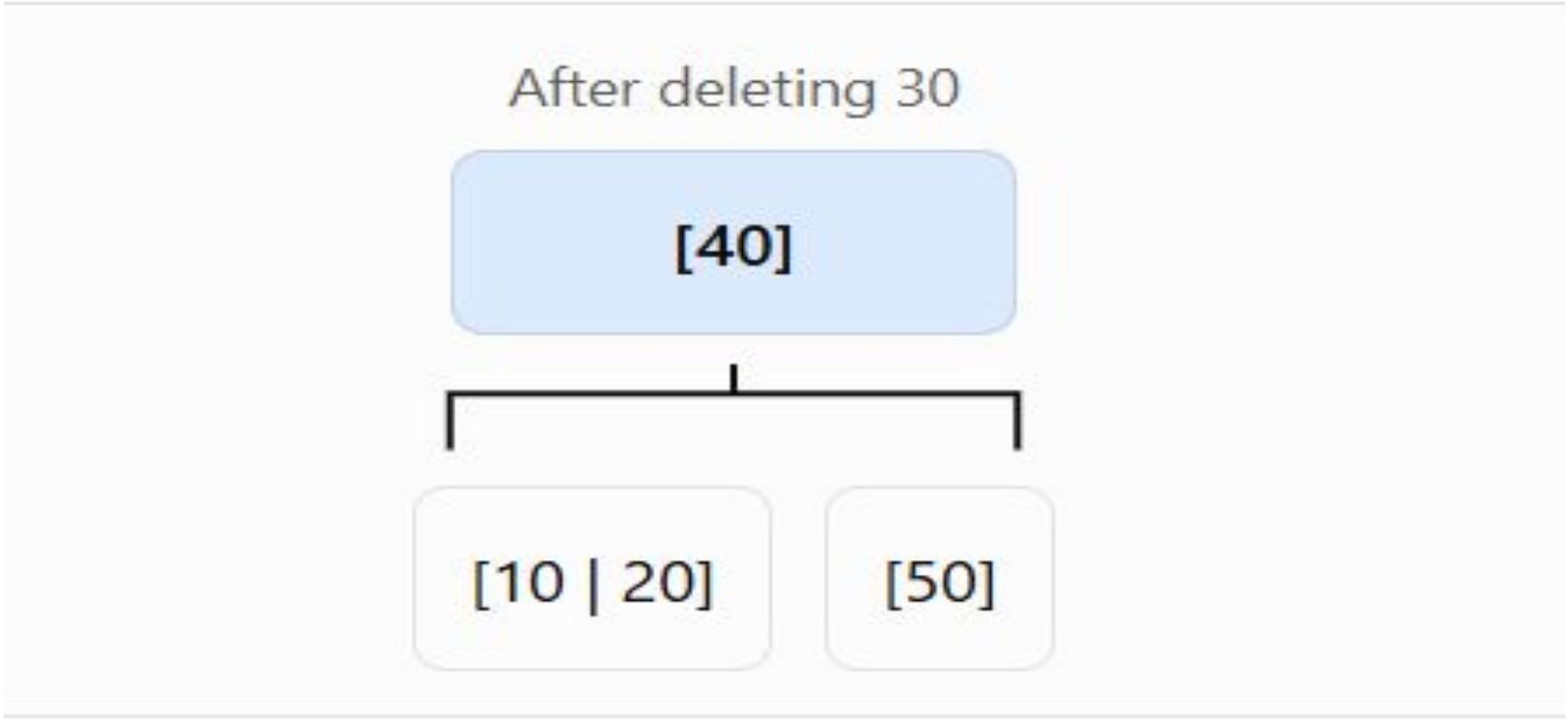
- The key 70 is in the right leaf node.
- Delete 70.
- The right leaf becomes empty, violating the minimum of one key.
- Its left sibling [40] has only one key, so it cannot lend a key.
- Merge the right child with its separator key 50 from the root.
- The resulting tree is:



Step 3: Delete 30

- The key 30 is in the root, an internal node.
- Replace 30 with its in-order successor, 40.
- Delete 40 from the right leaf [40 | 50].
- The right leaf becomes [50], which still satisfies the minimum of one key.

The final tree is:



B-Tree vs. B+ Tree

Feature	B-Tree	B+ Tree
Record pointers	Can be in internal and leaf nodes	At leaf level
Search may end	At an internal node or leaf	At a leaf
Key duplication	Generally keys need not be repeated in leaves	Separator keys may also appear in leaves
Linked leaves	Not required	Typically linked
Range searching	Supported	Generally more efficient
Main advantage	Search can find a record in an internal node	Efficient sequential and range access

Applications of B-Trees

- Database indexing
- File systems and storage systems
- Efficient retrieval of large amounts of sorted data
- Disk-based search structures
- A B-Tree is a balanced multiway search tree in which keys and record pointers may be stored in internal and leaf nodes. When a node overflows, it splits and promotes a separator key to its parent. This maintains balance and efficient searching.